Jac of All trades Graph-Theoretic Specification

Graph-Theoretic Specification

Implementation-Agnostic Design

This document defines the complete system as a **directed, functional graph** that is independent of any specific programming language or runtime. It is engineered to be directly realizable in **Jaseci/Jac** (nodes → `node`, edges → `++>` / `->` / `?:`, walkers → pure functional traversals, `by llm()` → typed decision vertices).

The design prioritizes referential transparency, explicit data flows, and native graph operations, making it ideal for agentic reasoning over portfolio state while satisfying the Agent Client Protocol (ACP) and Model Context Protocol (MCP).

System Architecture

The system follows a three-tier architecture:

1. **Front End ("JOAT"berg terminal)**: Replacing the role of DataSpell for user input and visualization. Acts as the ACP client.
2. **JOAT (Jac of all Trades) ACP agent**: The central orchestrator, implementing the ACP server and MCP client logic.
3. **Alpaca MCP server**: The tool provider for trading and market data, communicating with the JOAT ACP agent via **Interprocess Communication (IPC)**.

Graph Definition

```
G = (V, E, L)
```

- **V**: Finite set of nodes (entities with state and attributes).
- **E**: Finite set of directed edges (explicit, typed relationships).
- **L**: Labeling function that assigns attributes, types, and metadata to nodes and edges.

The graph is **cyclic at the system level** (for continuous agentic loops) but **directed-acyclic within individual walker traversals** (enforced via visited flags and functional updates). All operations are **pure functions** over graph sub-structures: walkers consume a source node/sub-graph, compute, and produce a new immutable result that is attached via explicit edges.

Nodes (V) — Core Entities

Node Type	Description	Key Attributes (L)	Jaseci Mapping (for reference)
JOATberg terminal	Front End (ACP Client) — source of user prompts and visualization sink	session_id, user_input, terminal_state	External entry node
JOAT ACP agent	Central Jaseci agent — implements ACP server and MCP client	capabilities, sessions (Dict[str, SessionState]), handlers, core logic	Root node (`node JOATAgent`)
AlpacaMCP-Server	Official Alpaca MCP server (tool provider)	tool_registry, auth_config, IPC_socket	External tool node
PortfolioGraph	Internal persistent Jaseci graph — core state model	last_updated, risk_score, total_value	Internal root (`node PortfolioRoot`)
Asset	Individual security (stock, crypto, option)	symbol, asset_type, current_price, market_data	`node Asset`
Position	User's holding in an asset	quantity, avg_cost_basis, unrealized_pnl	`node Position`
TradeEvent	Historical or proposed trade record	timestamp, action (buy/sell), quantity, price, status	`node TradeEvent`
RiskNode	Aggregated risk metrics or correlation vertex	volatility, correlation_matrix, exposure	`node RiskNode`
DecisionVertex	Temporary node created by `by llm()` for reasoning steps	prompt_type, confidence, rationale	Transient walker-generated node
VisualizationSink	Structured data payload for JOATberg rendering	table_data, plot_json, terminal_output	Output node attached to JOAT ACP agent

Edges (E) — Explicit Directed Connections

All edges are **typed** and carry semantic labels. Walkers traverse them functionally (no mutation of source; new edges are created for updated state).

Source Node	Target Node	Edge Type / Label	Direction	Purpose / Semantics
JOATberg terminal	JOAT ACP agent	`ACP_REQUEST`	→	JSON-RPC initialize / session/new / prompt
JOAT ACP agent	JOATberg terminal	`ACP_RESPONSE`	←	Serialized results + visualization payload

Source Node	Target Node	Edge Type / Label	Direction	Purpose / Semantics
JOAT ACP agent	AlpacaMCP-Server	`MCP_IPC_CALL`	□	Interprocess communication (get_portfolio, place_order)
JOAT ACP agent	PortfolioGraph	`HAS_PORTFOLIO`	++>	Root ownership of entire portfolio sub-graph
PortfolioGraph	Asset	`CONTAINS_ASSET`	++>	Portfolio → individual securities
Asset	Position	`HAS_POSITION`	++>	Asset → user's current holding
Position	TradeEvent	`HAS_TRADE_HISTORY`	-→	Position → chronological events
TradeEvent	Asset	`AFFECTS`	-→	Trade impacts asset state
PortfolioGraph	RiskNode	`HAS_RISK_PROFILE`	++>	Global risk metrics attached to portfolio
DecisionVertex	Asset / Position	`RECOMMENDS` / `EXECUTES`	-→	`by llm()` reasoning → concrete action
JOAT ACP agent	VisualizationSink	`GENERATES_VIZ`	-→	Final output for JOATberg terminal

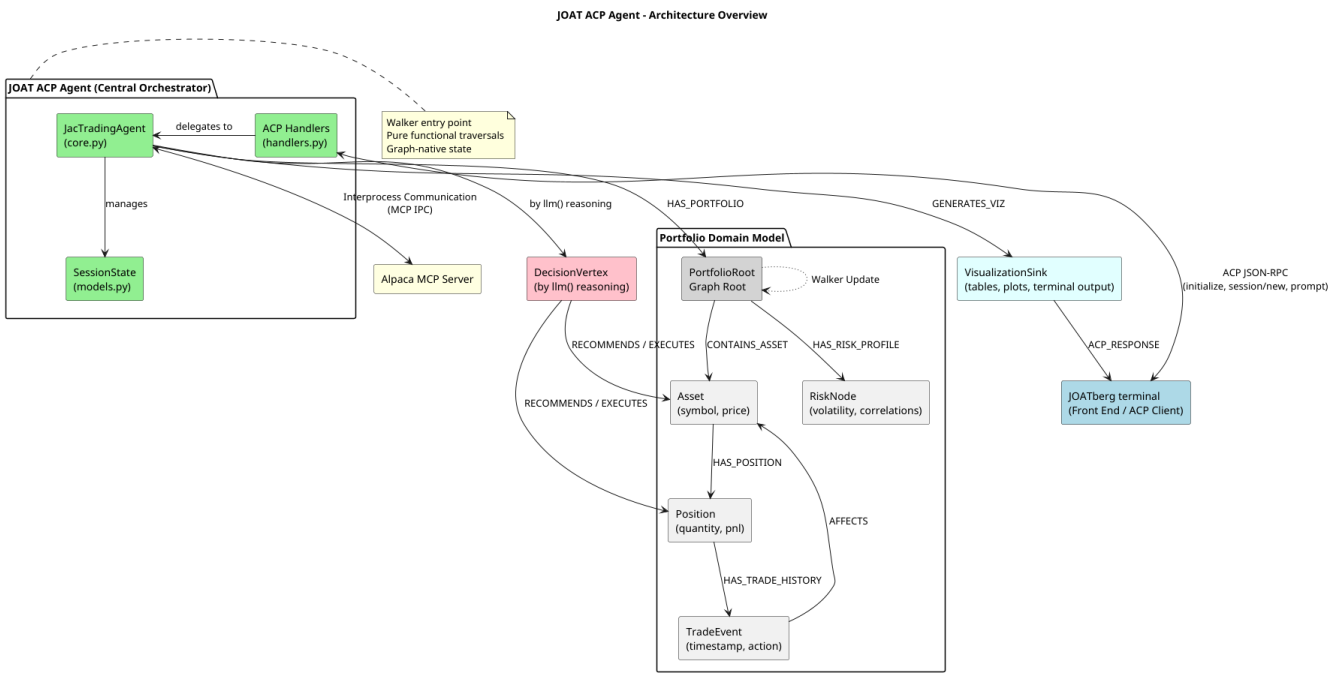
Functional Invariant: Every edge creation is a pure operation returning a new edge object; the original graph remains immutable until a walker explicitly commits an update via a new edge.

Labels & Metadata (L)

- **Node Labels**: Each node carries typed attributes (e.g., `symbol: str`, `quantity: float`, `risk_score: float`).
- **Edge Labels**: Semantic verbs + constraints (e.g., `MCP_TOOL_CALL` includes `tool_name`, `parameters`, `permission_required`).
- **Walker Metadata**: Each walker declares input/output types and guarantees purity (no side effects except controlled external protocol calls).
- **Session Labels**: Transient labels on `JOAT ACP agent` for ACP session state (e.g., `session_id`, `mcp_config_passed_by_terminal`).

Visual Graph (PlantUML)

The following diagram uses PlantUML syntax (component + deployment style for clarity, with explicit flows). It renders the same structure as the original Mermaid diagram but with PlantUML's powerful layout engine.


JOAT ACP Agent - Architecture Overview
JOAT ACP Agent (Central Orchestrator)
JacTradingAgent
(core.py)
delegates to
ACP Handlers
(handlers.py)
Walker entry point
Pure functional traversals
Graph-native state
manages
SessionState
(models.py)
Interprocess Communication
(MCP IPC)
Alpaca MCP Server
by llm() reasoning
DecisionVertex
(by llm() reasoning)
HAS_PORTFOLIO
Portfolio Domain Model
PortfolioRoot
Graph Root
Walker Update
GENERATES_VIZ
VisualizationSink
(tables, plots, terminal output)
ACP JSON-RPC
(initialize, session/new, prompt)
RECOMMENDS / EXECUTES
CONTAINS_ASSET
HAS_RISK_PROFILE
ACP_RESPONSE
RECOMMENDS / EXECUTES
Asset
(symbol, price)
RiskNode
(volatility, correlations)
JOATberg terminal
(Front End / ACP Client)
HAS_POSITION
AFFECTS
Position
(quantity, pnl)
HAS_TRADE_HISTORY
TradeEvent
(timestamp, action)